Estructuras de Datos, 2026-2

Proyecto del curso

Avance: lunes 2 de noviembre de 2026, 23:59
Entrega final: lunes 16 de noviembre de 2026, 23:59
Sustentaciones: 18 y 25 de noviembre

De qué se trata

El proyecto se hace en grupos de **hasta tres estudiantes** y pide resolver un problema real con dos estructuras distintas, las dos escritas desde cero, midiendo cuál cuesta menos y por qué. Vale el 30 % del curso: el avance 10 y la entrega final con sustentación 20.

Cada escenario nombra dos estructuras que resuelven el mismo problema con costos distintos. La primera es lineal y se entrega en el avance; la segunda aprovecha la organización de los datos y se entrega al final, junto con la medición de las dos. El contenedor de referencia, el de la biblioteca estándar que hace el mismo trabajo, entra solo como tercera curva de la gráfica, para ver dónde queda una implementación de producción.

Cómo se escoge el escenario

El escenario se reserva en el formulario que se publica en la página del proyecto:

`https://cardel.github.io/notasUniversidad/2026-II/Estructuras%20de%20Datos/Proyecto/Proyecto%20del%20curso/`

Lo llena un integrante por grupo, con los nombres y códigos de todos. Cada escenario admite **cuatro grupos**; cuando se completa, deja de aparecer en la lista. Se reserva por orden de llegada y el formulario cierra cuando se ocupan los dieciséis cupos.

Los cuatro escenarios

Los cuatro leen órdenes de la entrada estándar y escriben en la salida estándar, una línea por cada orden que produzca respuesta. La primera línea dice cuántas órdenes vienen. Las dos estructuras responden lo mismo a la misma entrada, salvo donde el escenario diga otra cosa: eso es lo que permite medirlas con el mismo programa y comparar sus salidas.

A. Inventario de una tienda

Cada artículo tiene código numérico, nombre y cantidad.

Avance: lista enlazada simple, con búsqueda lineal por código. *Final:* tabla hash con encadenamiento y clave entera, donde cada cubeta es la lista del avance. *Referencia:* `map`.

Entrada. `AGREGAR c nombre cantidad`, `ELIMINAR c`, `CONSULTAR c`, `REPONER c cantidad` y `LISTAR`. El nombre es una palabra sin espacios.

Salida. `CONSULTAR` escribe `c nombre cantidad`, o `no existe`. `LISTAR` escribe un artículo por línea. Las otras tres escriben `ok`, o `no existe` cuando el código no está, o `repetido` cuando se agrega uno que ya estaba.

Entrada	*Salida*

`7`
`AGREGAR 1004512 arroz 20`
`AGREGAR 1004498 azucar 5`
`CONSULTAR 1004512`
`REPONER 1004498 10`
`CONSULTAR 1004498`
`ELIMINAR 1004512`
`CONSULTAR 1004512`
	`ok`
	`ok`
	`1004512 arroz 20`
	`ok`
	`1004498 azucar 15`
	`ok`
	`no existe`

Lo que se mide. El tiempo de consulta contra el número de artículos, con tres tamaños de tabla (uno pequeño, uno cercano al número de artículos y uno primo mayor) y un cuarto juego de códigos escogido para que todos caigan en la misma cubeta. Expliquen en qué orden sale `LISTAR` con cada estructura y qué hay que hacer si se quiere el mismo orden en las dos.

B. Frecuencias de palabras de un texto

Se cuenta cuántas veces aparece cada palabra distinta. Una palabra es una tira maximal de letras, y mayúsculas y minúsculas no se distinguen: todo se pasa a minúscula y los signos de puntuación separan.

Avance: lista enlazada con un contador en cada nodo, donde insertar o incrementar recorre buscando la palabra. *Final:* tabla hash con encadenamiento y clave de cadena; la función hash la diseña el grupo y la justifica. *Referencia:* `map`.

Entrada. La primera línea trae k. Lo que sigue, hasta el fin del archivo, es el texto.

Salida. Las k palabras más frecuentes, una por línea, `palabra cuenta`, de mayor a menor cuenta; entre palabras con la misma cuenta, en orden alfabético.

Entrada	*Salida*

`2`
`El gato y el perro.`
`El gato duerme.`
	`el 3`
	`gato 2`

Las tres apariciones de `el` se cuentan juntas aunque una vaya en mayúscula, y el punto no entra en la palabra `perro`.

Lo que se mide. Cómo quedan repartidas las cubetas con un texto de verdad (la más larga, el promedio y cuántas quedan vacías) al variar el tamaño de la tabla y el factor de carga, y el tiempo de consulta en cada caso.

C. Índice de títulos

Un catálogo de títulos de libros o de canciones que se consulta por nombre.

Avance: lista enlazada ordenada, donde insertar deja el título en su lugar y buscar se detiene al pasarse. *Final:* árbol binario de búsqueda, con el borrado en sus tres casos (hoja, un hijo, dos hijos) y el recorrido en orden. *Referencia:* `set`.

Entrada. `INSERTAR t`, `BUSCAR t`, `ELIMINAR t` y `LISTAR`. El título es el resto de la línea y puede llevar espacios.

Salida. `BUSCAR` escribe `si` o `no`. `LISTAR` escribe los títulos en orden alfabético, uno por línea. `INSERTAR` y `ELIMINAR` escriben `ok`, o `repetido` y `no existe`.

Entrada	*Salida*
7	ok
INSERTAR Cien anos de soledad	ok
INSERTAR Altazor	si
BUSCAR Altazor	ok
INSERTAR Pedro Paramo	Altazor
LISTAR	Cien anos de soledad
ELIMINAR Altazor	Pedro Paramo
BUSCAR Altazor	ok
	no

Lo que se mide. La altura del árbol según el orden en que entran los títulos, comparando una entrada revuelta contra una entrada ya alfabetizada, y el tiempo de búsqueda en los dos casos. Con los títulos alfabetizados el árbol degenera: digan en qué se convierte y qué le pasa entonces a las dos curvas de la gráfica.

D. Turnos de una clínica

Los pacientes llegan con un nivel de urgencia de 1 a 5, donde 5 es el más urgente; esperan y se atienden. El reloj avanza una unidad por orden leída, y lo que esperó un paciente es la diferencia entre la orden que lo atendió y la que lo registró.

Avance: cola circular sobre arreglo, que atiende por orden de llegada. *Final:* montículo binario sobre arreglo, con flotar y hundir, que atiende primero al más urgente. *Referencia:* `priority_queue`.

Entrada. `LLEGA nombre urgencia`, `ATIENDE`, `ESPERAN` y `PROMEDIO`.

Salida. `ATIENDE` escribe el nombre del paciente que pasa, o `nadie espera`. `ESPERAN` escribe cuántos hay en la sala. `PROMEDIO` escribe con un decimal lo que esperaron en promedio los ya atendidos. `LLEGA` no escribe nada.

Este es el único escenario donde las dos estructuras responden distinto a la misma entrada, y esa diferencia es el resultado:

Entrada	*Salida, cola circular*	*Salida, montículo*
7	2	2
LLEGA ana 2	ana	beto
LLEGA beto 5	beto	ana
ESPERAN	0	0
ATIENDE	3.0	3.0
ATIENDE		
ESPERAN		
PROMEDIO		

El promedio sale igual en los dos, 3,0, porque las dos atienden a los mismos dos pacientes en las mismas dos órdenes. Lo que cambia es quién espera: con la cola circular el urgente esperó 3 y con el montículo esperó 2.

Lo que se mide. El tiempo de espera de los urgentes bajo los dos esquemas, alimentando los dos con la misma secuencia de llegadas, y el costo de registrar y de atender en cada uno. Expliquen el resultado: una de las dos cuesta más por operación y aún así atiende mejor a quien debe.

Las dos estructuras se escriben con arreglos, punteros o nodos, según corresponda, y **ninguna de las dos**

usa contenedores de la biblioteca estándar por dentro. El contenedor de referencia se usa tal cual, solo para medirlo.

Qué se entrega

El trabajo vive en un repositorio de GitHub, uno por grupo, que sale de *fork* del repositorio base: `https://github.com/EjerciciosClasesCardel/estr-2026-2-proyecto`.

1. Un integrante le hace fork; el fork queda público y las dos entregas se hacen sobre él. **Una entrega que no sea fork de ese repositorio pierde el 30 % de la nota.**
2. En `Settings > Collaborators` agrega a los demás integrantes, para que cada uno confirme con su propia cuenta.
3. Llenan el `README.md` con los integrantes y el escenario reservado.

Se califica el último commit anterior a la hora de cierre de cada entrega; lo que se suba después no se tiene en cuenta. La historia de commits cuenta doble: dice si hubo avance (un repositorio con un solo commit el día del cierre no lo muestra) y dice quién hizo qué. Los cambios tienen que ser **significativos**: los de puro formato o de comentarios no cuentan. **Cada integrante confirma con su propia cuenta**; un proyecto donde todo lo subió una sola persona se sustenta igual, pero la nota individual puede quedar distinta.

Avance — lunes 2 de noviembre, 23:59

1. **Informe corto** en `docs/informe-avance.md`: el escenario descrito en sus propias palabras; qué guarda la estructura y qué operaciones necesita; por qué la estructura lineal escogida y qué costo esperan de cada operación, citando capítulo y sección del texto guía; y qué piensan medir al final.
2. **La estructura lineal del escenario**, escrita desde cero, con todas sus operaciones funcionando (en `codigo/`).
3. **Casos de prueba**: al menos tres funcionales y uno límite (estructura vacía, un solo elemento, estructura llena, clave repetida), con la salida esperada y la obtenida.
4. **Bitácora** (`bitacora.md`): una entrada fechada por sesión de trabajo, con qué hizo y qué quedó pendiente.

Entrega final — lunes 16 de noviembre, 23:59

1. **Las dos estructuras** resolviendo el mismo problema con las mismas operaciones y las mismas firmas, de modo que el programa que mide pueda usar una o la otra sin cambiar.
2. **Análisis formal** del costo en tiempo y en espacio de cada operación de las dos, con la estructura de cuatro partes: teorema, estrategia, desarrollo y conclusión con testigos c y k. Los invariantes se escriben como fórmulas.
3. **Experimentos**: midan las dos estructuras con entradas de tamaños crecientes, al menos cinco tamaños, en la misma máquina y con el mismo compilador. Agreguen el contenedor de referencia como tercera curva. Reporten los tiempos en una tabla y en una gráfica de Mermaid, y digan si lo medido coincide con lo que predijo el análisis; cuando no coincida, expliquen por qué.
4. **Informe final** en `docs/informe-final.md`: el del avance corregido, más el análisis, los experimentos y las conclusiones. Las estructuras se dibujan con Mermaid.

5. **Sustentación** de 10 minutos: se explica el modelado, se muestra el código corriendo y se responden preguntas sobre las decisiones y los costos.

Reglas del repositorio

1. **Solo archivos de texto plano**: código fuente (`.c`, `.cpp`, `.h`), documentación (`.md`), datos de prueba (`.in`, `.out`, `.csv`) y el `Makefile`. No se aceptan comprimidos (`.zip`, `.rar`, `.tar.gz`) ni binarios (`.pdf`, `.docx`, `.xlsx`, ejecutables, imágenes escaneadas o insertadas). Los archivos de compilación se ignoran con `.gitignore`.
2. El `README.md` lleva el **nombre, el código y el correo institucional** de cada integrante, y el escenario reservado. Si el archivo falta, o si le falta alguno de esos datos, la entrega pierde el **20 %** de la nota. Quien no aparezca ahí no cuenta como integrante y su nota del proyecto es 0,0.
3. Las **primeras líneas de cada archivo de código** llevan los autores con su código: `// Autores: Nombre1 Codigo1, Nombre2 Codigo2`
4. Los informes van en **Markdown** dentro de la carpeta `docs/`, con los nombres `informe-avance.md` e `informe-final.md`. No se acepta otro formato ni archivos por fuera de esa carpeta.
5. La notación matemática se escribe en LaTeX dentro del Markdown, con los delimitadores `$...$` y `$$...$$`: las cotas, los testigos y los invariantes van así. No se aceptan imágenes de fórmulas.
6. **Todos los diagramas** — nodos y punteros de la estructura, árboles, tablas con sus colisiones, trazas de una operación, y la gráfica de los tiempos medidos — se hacen con **Mermaid** dentro del Markdown. La gráfica de los experimentos usa `xychart-beta`. No se aceptan imágenes insertadas ni diagramas escaneados.

Las tres guías de GitHub, en el orden de las reglas 4, 5 y 6:

- `https://docs.github.com/es/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax`
- `https://docs.github.com/es/get-started/writing-on-github/working-with-advanced-formatting/writing-mathematical-expressions`
- `https://docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams`

Cómo se califica

Las dos entregas se califican sobre 100 puntos y la nota sale de $5 \times \text{puntos}/100$. Cada criterio trae los niveles con los que se asigna su puntaje; el nivel intermedio es el que se usa cuando lo pedido está pero incompleto. Todos los criterios son del grupo salvo la sustentación, que se pregunta a cada integrante y se califica por separado: la nota de un proyecto puede quedar distinta entre los tres si el trabajo o el dominio del tema lo estuvo.

Avance — 100 puntos

Modelado del escenario — 30 puntos.

30 el escenario está descrito en sus palabras, con qué guarda la estructura y qué operaciones necesita; la elección de la estructura lineal está justificada frente a las otras, y los costos esperados aparecen con la razón de cada uno y con la cita al capítulo y la sección del texto guía.

21 están el escenario, las operaciones y los costos, pero la justificación no dice por qué esa estructura y no otra, o falta la cita.

12 el escenario está copiado del enunciado y los costos aparecen sin razón que los sostenga.

0 no hay informe de avance, o no dice qué estructura se escogió.

Implementación de la estructura lineal — 40 puntos.

40 todas las operaciones del escenario corren; el código respeta las restricciones de la última sección, libera toda la memoria que reserva y compila sin advertencias con `-Wall -Wextra`.

28 las operaciones corren, pero una falla en un caso límite, o quedan advertencias del compilador, o hay memoria reservada que no se libera.

16 compila y corre parte de las operaciones, o delega en la biblioteca estándar lo que debía escribirse desde cero.

0 no compila.

Casos de prueba — 20 puntos.

20 al menos tres funcionales y uno límite, cada uno con la entrada, la salida esperada y la obtenida, y `make pruebas` los corre.

14 los casos están pero falta el límite, o no aparece la salida obtenida.

8 hay pruebas sueltas sin salida esperada, o no se pueden correr.

0 no hay casos de prueba.

Bitácora — 10 puntos.

10 una entrada fechada por sesión de trabajo, con qué se hizo y qué quedó pendiente, y las fechas se corresponden con la historia de commits.

7 las entradas están, pero algunas no dicen qué quedó pendiente o no coinciden con los commits.

4 dos o tres entradas para todo el periodo.

0 no hay bitácora.

Entrega final — 100 puntos

Las dos estructuras con la misma interfaz — 30 puntos.

30 las dos resuelven el escenario completo con las mismas operaciones y las mismas firmas, el programa que mide usa una o la otra sin cambiar, y las dos respetan las restricciones y liberan la memoria.

21 las dos corren, pero la interfaz no es la misma y el programa que mide las trata distinto, o una falla en un caso límite.

12 solo una de las dos está completa.

0 el código no compila.

Análisis formal de complejidad — 20 puntos.

20 cada operación de las dos estructuras lleva su costo en tiempo y en espacio, demostrado en cuatro partes (teorema, estrategia, desarrollo y conclusión con testigos c y k), y los invariantes se escriben como fórmulas con sus cuantificadores y sus rangos.

14 los costos y las demostraciones están, pero algún paso se da por obvio o los invariantes están escritos como frases.

8 los costos se declaran sin demostrarlos.

0 no hay análisis.

Experimentos — 20 puntos.

20 al menos cinco tamaños crecientes, en la misma máquina y con el mismo compilador, con la tabla de tiempos, la gráfica en `xychart-beta` y la curva del contenedor de referencia; el texto dice si lo medido coincide con lo que predijo el análisis y explica las diferencias.

14 las mediciones y la gráfica están, pero el contraste contra el análisis no aparece o se queda en afirmar que coincide.

8 hay mediciones de un solo tamaño, o no hay gráfica.

0 no hay experimentos.

Informe — 15 puntos.

15 está en `docs/informe-final.md`, recoge corregido el del avance, dibuja las estructuras con Mermaid, escribe la notación en LaTeX y cita el texto guía con capítulo y sección.

10 está completo, pero describe las estructuras en palabras en vez de dibujarlas, o no cita.

6 es el del avance sin corregir, o le falta el análisis o los experimentos.

0 no está en `docs/` o no está en Markdown.

Sustentación, calificada por integrante — 15 puntos.

15 explica el modelado sin leerlo, muestra corriendo la parte que le tocó y, ante una operación que se le señale sin aviso, dice qué cuesta y de dónde sale ese costo; si se le cambia algo del escenario, dice qué habría que tocar de la estructura.

10 explica el modelado y muestra el código corriendo, y responde por los costos que trajo preparados, pero no por el de una operación que se le señale sin aviso.

6 cuenta lo que hizo el grupo sin dar razón de las decisiones ni de los costos, o no logra mostrar corriendo la parte que le tocó.

0 no se presenta a la sustentación.

Quien no se presenta a la sustentación tiene 0,0 en la entrega final, aunque el resto del grupo sustente. Cuando la sustentación queda por debajo de 10 puntos, los puntos de los demás criterios le cuentan a esa persona en proporción a lo que sustentó: se multiplican por $S/15$, donde S es su puntaje de sustentación. Con 6 puntos de sustentación y 80 del grupo, por ejemplo, la persona queda con $0{,}4 \times 80 + 6 = 38$ puntos y una nota de 1,9. Un proyecto cuyo código no compila no recibe los puntos de implementación ni los de experimentos.

Situación	Penalización
El repositorio entregado no es fork del repositorio base	30 % de la nota
El `README.md` falta o le falta el nombre, el código o el correo de algún integrante	20 % de la nota
Hay comprimidos o binarios en el repositorio	10 % de la nota
El informe no está en `docs/` y en Markdown	no recibe los puntos de informe

Restricciones de código

Las mismas del curso, y se califican:

- C o C++, compilado con `gcc -Wall -Wextra` o `g++ -Wall -Wextra`, sin advertencias.
- Solo biblioteca estándar. Las dos estructuras escritas desde cero no usan contenedores de la STL por dentro; el contenedor de referencia se usa tal cual, solo para medirlo.
- Sin `break`, sin `continue`, sin `goto`, y un solo `return` al final de cada función.
- Toda reserva con `malloc` o `new` va con su liberación, y el retorno de `malloc` se verifica.
- El código compila con el `Makefile` del repositorio: `make` produce los ejecutables y `make pruebas` corre los casos.

Integridad

Dentro del grupo se comparte todo; entre grupos, no. Las entregas se comparan con herramientas de similitud y con un detector de patrones de código generado; lo escrito íntegramente por inteligencia artificial se considera plagio y puede derivar en un proceso ante la Dirección de Programa. Citen en el informe lo que consultaron.